

Proyecto segundo corte

Edwin Stiven Sosa Bohorques*, Juan Sebastian Zapata Garcia†

*Universidad Santo Tomas

Email: edwardsosa@usantotomas.edu.co

†Universidad Santo Tomas

Email: juan.zapata@usantotomas.edu.co

I. OBJETIVO

Configurar una VM Debian anidada que funcione como hipervisor, crear una subred interna y desplegar en ella dos sub-VMs (Arch y Rocky) y dos contenedores Docker (Garuda y Fedora), de forma que la comunicación se mantenga exclusivamente dentro de la VM Debian.

II. ENTORNO Y PREREQUISITOS

- Host físico: Ubuntu con soporte AMD-V (svm) y QEMU/KVM instalado.
- ISOs requeridas: Debian (invitado hipervisor), Arch, Rocky.
- Permisos para ejecutar QEMU con `-enable-kvm` y `-cpu host`.
- Paquetes dentro de Debian: `qemu-kvm`, `libvirt-daemon-system`, `bridge-utils`, `docker.io`.

III. PROCEDIMIENTO

A continuación se listan los pasos principales en orden cronológico.

A. Habilitar virtualización anidada en el host (AMD)

```
# Habilitar nested temporalmente
echo 1 | sudo tee /sys/module/kvm_amd/parameters/nested

# Persistente
sudo bash -c 'echo "options kvm_amd nested=1" > /etc/modprobe.d/kvm_amd.conf'
sudo modprobe -r kvm_amd
sudo modprobe kvm_amd

# Verificación
cat /sys/module/kvm_amd/parameters/nested
```

B. Iniciar la VM Debian desde el host con passthrough de CPU

```
qemu-system-x86_64 \
  -enable-kvm -cpu host -m 8192 -smp 4 \
  -drive file=~/.vms/debian-nested.qcow2,format=qcow2 \
  -cdrom /ruta/a/debian.iso -boot d \
  -netdev bridge,id=net0,br=br0 \
  -device virtio-net-pci,netdev=net0 \
  -vga virtio
```

C. Preparación dentro de la VM Debian (hipervisor)

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system
  bridge-utils docker.io
sudo systemctl enable --now docker
```

D. Crear la subred interna (bridge) en Debian

```
sudo ip link add name vmnet0 type bridge
sudo ip addr add 10.10.0.1/24 dev vmnet0
sudo ip link set vmnet0 up
```

E. Crear y arrancar las sub-VMs (Arch y Rocky)

```
# crear discos
qemu-img create -f qcow2 arch.qcow2 20G
qemu-img create -f qcow2 rocky.qcow2 20G

# arrancar instalador Arch
qemu-system-x86_64 -enable-kvm -cpu host -m 2048 \
  -drive file=arch.qcow2,format=qcow2 \
  -cdrom /ruta/a/arch.iso -boot d \
  -netdev bridge,id=net0,br=vmnet0 \
  -device virtio-net-pci,netdev=net0

# arrancar instalador Rocky (an logo)
```

Asignar direcciones IPv4 estáticas en la subred 10.10.0.0/24 (ej.: Arch 10.10.0.2, Rocky 10.10.0.3).

F. Crear la red Docker (macvlan) para compartir la misma L2

```
docker network create -d macvlan \
  --subnet=10.10.0.0/24 \
  --gateway=10.10.0.1 \
  -o parent=vmnet0 \
  dockervm_net

# desplegar contenedores con IPs fijas
docker run -dit --name garuda --network dockervm_net
  --ip 10.10.0.10 archlinux sh
docker run -dit --name fedora --network dockervm_net
  --ip 10.10.0.11 fedora sh
```

G. Crear interfaz macvlan sibling en Debian

```
sudo ip link add mac0 link vmnet0 type macvlan mode
  bridge
sudo ip addr add 10.10.0.254/24 dev mac0
sudo ip link set mac0 up
```

TABLE I: Direcciones IPv4 de ejemplo

Entidad	Dirección IPv4
Debian (bridge vmnet0)	10.10.0.1/24
Sub-VM Arch	10.10.0.2/24
Sub-VM Rocky	10.10.0.3/24
Contenedor Garuda	10.10.0.10/24
Contenedor Fedora	10.10.0.11/24
Interfaz macvlan sibling (Debian)	10.10.0.254/24

IV. DIRECCIONES IPV4 DE EJEMPLO

V. COMPROBACIONES

```

# en Debian (hipervisor)
ip addr show vmnet0
docker network ls
docker network inspect dockervm_net

# desde una sub-VM (ej. Arch)
ping -c3 10.10.0.10 # hacia contenedor Garuda

# desde un contenedor
docker exec -it garuda ping -c3 10.10.0.2 # hacia
Arch

```